

Dựng môi trường Benchmark PQC

WP1 — Giải thích 12 file, theo từng file
trên **x86_64** và **ARM — Raspberry Pi 4 (Mythic Beasts)**

Đồ án NT219 · So sánh ML-KEM / ML-DSA với RSA / ECC
(Kyber, Dilithium) — Linux x86_64 và ARM aarch64

Contents

1 Tổng quan: vì sao có 12 file và hai máy	2
2 Bản đồ: file nào sinh ra gì, đặt ở đâu	2
3 <code>versions.env</code> — <code>scripts/</code>	3
4 <code>setenv.sh</code> — <code>scripts/</code>	3
5 <code>install_prereqs.sh</code> — <code>scripts/</code>	4
6 <code>build_openssl.sh</code> — <code>scripts/</code>	4
7 <code>build_liboqs.sh ref opt</code> — <code>scripts/</code>	5
8 <code>fetch_pqclean.sh [clean avx2 aarch64]</code> — <code>scripts/</code>	6
9 <code>build_oqsprovider.sh [ref opt]</code> — <code>scripts/</code>	6
10 <code>cross_build_liboqs.sh</code> — <code>scripts/</code>	7
11 <code>aarch64-toolchain.cmake</code> — <code>cmake/</code>	7
12 <code>docker_buildx.sh</code> — <code>scripts/</code>	8
13 <code>Dockerfile.x86_64</code> — <code>docker/</code>	8
14 <code>verify_env.sh</code> — <code>scripts/</code>	9
15 Hồ sơ tái lập (thư mục <code>docs/</code>)	9
16 Quy tắc bốn con số khi so sánh	9
17 Phụ lục — Trình tự chạy đầy đủ, hai máy	9

1. Tổng quan: vì sao có 12 file và hai máy

WP1 là bước *dụng môi trường* — biến một máy Linux trắng thành nơi chạy được ML-KEM/ML-DSA và đo đạc chúng. Toàn bộ công việc gói trong 12 file. Điểm cốt lõi phải nắm trước khi đọc chi tiết: **x86_64 và ARM không phải “một việc làm hai lần” mà là hai thí nghiệm khác nhau.**

- **Máy x86_64** (laptop) trả lời **RQ1**: PQC tốn kém thế nào so với RSA/ECC trên một server điển hình. Đây cũng là nơi phát triển, gỡ lỗi, và là nơi *duy nhất* chạy được nhóm file bằng chúng cross-compile/Docker.
- **Máy ARM — Raspberry Pi 4** trả lời **RQ2**: tối ưu NEON có kéo PQC về mức khả thi trên thiết bị nhúng không. Pi 4 dùng CPU **Cortex-A72** (SoC BCM2711), 4 nhân 1.5 GHz, ARMv8-A 64-bit.

Ba nguyên tắc vàng xuyên suốt tài liệu:

1. **Binary là tài sản theo máy.** Mọi thứ trong ~/pqc build trên máy nào chỉ chạy trên máy đó; bản tối ưu còn gắn chặt với đúng con CPU đã build ra nó. Không bao giờ copy ~/pqc chéo máy.
2. **Số đo là tài sản theo máy.** Chỉ so trực tiếp hai số đo *cùng một máy*. Muốn so giữa hai máy thì so *tỉ lệ* (xem mục 16), không so số thô.
3. **Một bộ script cho cả hai máy.** Không có “bản script cho ARM” riêng. Cùng repo, cùng lệnh; script tự nhận diện kiến trúc qua uname -m và tự đổi cờ. Khác biệt nằm ở *kịch bản vận hành*, là thứ tài liệu này mô tả.

► Cách đọc

Mỗi file ở các mục sau đều có cùng năm phần: *Nhiệm vụ* (làm gì trong đồ án) — *Sinh ra gì & ở đâu* (tạo ra file nào, đặt đâu) — *Trên x86_64* — *Trên ARM Pi 4* — *Lưu ý*. File nào hai máy giống nhau sẽ gộp thành một phần “Trên cả hai máy” để khỏi dài dòng.

2. Bản đồ: file nào sinh ra gì, đặt ở đâu

Mọi thứ cài vào ~/pqc (**không** dụng OpenSSL hệ thống). Sau khi chạy đủ, cây thư mục như sau:

```
~/pqc/
|-- openssl/           <- build_openssl.sh      (OpenSSL 3.6.2 + PQC native)
|-- liboqs-ref/        <- build_liboqs.sh ref    (C thuan, NEON OFF)
|-- liboqs-neon/       <- build_liboqs.sh opt    (tối ưu: NEON/AVX2 ON)
|-- liboqs-aarch64-cross/ <- cross_build_liboqs.sh (CHI trên x86; bang chung)
|-- oqs-provider/      <- build_oqsprovider.sh (provider cho TLS)
`-- src/               <- nguồn đã clone (tại dụng, không build lại)
    |-- openssl/
    |-- liboqs/
    |   |-- build-ref/tests/speed_kem, speed_sig
    |   |-- build-opt/tests/speed_kem, speed_sig
    |-- PQClean/crypto_kem|crypto_sign/<scheme>/<variant>/lib*.a
    `-- oqs-provider/
```

Đồng thời, bằng chứng tái lập tự gom vào docs/ trong repo:

```
<repo>/docs/
|-- openssl.commit  liboqs.commit  pqclean.commit  oqsprovider.commit
|-- env_report_x86_64.txt          <- verify_env.sh trên máy x86
`-- env_report_aarch64.txt         <- verify_env.sh trên máy ARM
```

File	Sinh ra	Ở đâu
versions.env	(không)	file đầu vào, được source
setenv.sh	biến môi trường	chỉ trong shell hiện tại
install_prereqs.sh	gói hệ thống	/usr
build_openssl.sh	OpenSSL + commit	~/pqc/openssl, docs/openssl.commit
build_liboqs.sh	2 thư viện + tools	~/pqc/liboqs-{ref,neon}, docs/liboqs.commit
fetch_pqclean.sh	các file .a	~/pqc/src/PQClean/.../lib*.a, docs/pqclean.commit
build_oqsprovider.sh	provider .so	~/pqc/oqs-provider/lib, docs/oqsprovider.commit
cross_build_liboqs.sh	binary ARM (cross)	~/pqc/liboqs-aarch64-cross
aarch64-toolchain.cmake	(không)	file cấu hình cho cross
docker_buildx.sh	image đa kiến trúc	trong Docker
Dockerfile.x86_64	image x86	trong Docker
verify_env.sh	biên bản môi trường	docs/env_report_<arch>.txt

3. versions.env — scripts/

Nhiệm vụ

Nguồn-sự-thật-duy-nhất của cả dự án: ghim phiên bản (OPENSSL_TAG=openssl-3.6.2, LIBOQS_TAG=0.14.0, OQS_PROVIDER_TAG=0.10.0) và mọi đường dẫn dưới ~/pqc. Mọi script khác đọc các biến này thay vì tự định nghĩa, nên đổi một phiên bản chỉ sửa một chỗ — đây là điểm chấm *Reproducibility* của đề bài.

Sinh ra gì & ở đâu

Không sinh ra gì. Nó là *đầu vào*, được các script nạp bằng source.

Trên cả hai máy

Giống hệt nhau — chỉ cần clone repo về là có. Không chạy trực tiếp.

Lưu ý

Muốn kiểm một biến có còn ai dùng không (trước khi xóa): `grep -rl '$VAR' --include='*.sh' .` — nếu ra 0 file thì đó là cấu hình chết, bỏ được.

4. setenv.sh — scripts/

Nhiệm vụ

Bật OpenSSL tự build cho *terminal hiện tại*: thêm ~/pqc/openssl/bin vào PATH và thư mục thư viện vào LD_LIBRARY_PATH. Cách này theo đúng quy ước demo chính thức của OpenSSL: link động, *không* nhúng rpath, tìm thư viện lúc chạy qua LD_LIBRARY_PATH.

Sinh ra gì & ở đâu

Chỉ sinh ra biến môi trường, sống trong shell hiện tại, không tạo file.

Trên cả hai máy

Giống hệt. Phải dùng source scripts/setenv.sh — **không** chạy ./setenv.sh.

Lưu ý

Đây là lỗi phổ biến nhất của người mới: chạy ./setenv.sh thì biến môi trường sống trong tiến trình con rồi biến mất; phải source để biến vào chính shell đang dùng. Kiểm: sau khi source, `which openssl` phải ra ~/pqc/openssl/bin/openssl. Mỗi cửa sổ terminal mới đều phải source lại.

5. install_prereqs.sh — scripts/

Nhiệm vụ

Cài toàn bộ công cụ: trình biên dịch (gcc, g++, cmake, ninja, perl, git), công cụ đo (/usr/bin/time, htop, apache2-utils cho ab, wrk, tshark, perf), và thư viện phân tích (pandas, matplotlib). **Đây là script duy nhất cần sudo.**

Sinh ra gì & ở đâu

Cài gói hệ thống vào /usr. Không tạo file trong repo.

Trên x86_64

Chạy thẳng: `bash scripts/install_prereqs.sh`. Giữa chừng hộp thoại cấu hình wireshark-common hiện ra hỏi cho phép user thường bắt gói tin — **chọn Yes** rồi Enter. Để quyền đó áp vào tài khoản, sau khi script xong chạy `sudo usermod -aG wireshark $USER` rồi đăng xuất/đăng nhập lại.

Trên ARM Pi 4 (Mythic Beasts)

Lệnh y hệt; apt tự kéo các gói bản arm64. Dòng `linux-tools-$(uname -r)` trong script phát huy tác dụng ở đây: kernel của Pi/cloud có hậu tố riêng, lấy đúng `uname -r` mới cài được perf khớp kernel (gói `linux-tools-generic` để lệch).

Lưu ý

`DEBIAN_FRONTEND=noninteractive` trong script *không* lọt qua sudo (sudo reset môi trường), nên hộp thoại tshark vẫn hiện — đây là chủ ý: một quyết định bảo mật nên được chọn có ý thức một lần. Tư cách thành viên nhóm wireshark chỉ được nạp khi đăng nhập lại (hoặc newgrp wireshark).

6. build_openssl.sh — scripts/

Nhiệm vụ

Build OpenSSL 3.6.2 có sẵn ML-KEM/ML-DSA ngay trong libcrypto (PQC “native”). Đây là **nền của WP2, WP4, WP5** — mọi phép đo bằng `bench_ev` và mọi thí nghiệm TLS đều dựa vào bản OpenSSL này.

Sinh ra gì & ở đâu

- Cài đặt: `~/pqc/openssl/` (có `bin/openssl`, `include/`, `lib` hoặc `lib64`).
- Nguồn: `~/pqc/src/openssl/` (giữ lại để tái dùng).
- Bằng chứng: `docs/openssl.commit` (commit chính xác đã build).

Trên x86_64

```
bash scripts/build_openssl.sh
```

Clone tag 3.6.2 (1 phút), Configure, make khoảng 15–30 phút, rồi test suite. Nên chạy đủ test **một lần** để lấy bằng chứng; các lần rebuild sau dùng `SKIP_TESTS=1`. Kết thúc phải thấy dòng `DONE` và `openssl version` in 3.6.2.

Trên ARM Pi 4 (Mythic Beasts)

Cùng lệnh, nhưng Pi 4 (4 nhân Cortex-A72, đọc/ghi thẻ microSD chậm) build **lâu hơn nhiều** — make có thể 30–60+ phút. Chiến thuật:

```
SKIP_TESTS=1 bash scripts/build_openssl.sh # ban ngay: nhanh, de lam viec tiep
# chay rieng MOT lan qua dem de co bang chung test pass:
FORCE=1 bash scripts/build_openssl.sh
```

Cảnh báo mạng IPv6-only: Pi tại Mythic Beasts chỉ có IPv6, trong khi GitHub chưa hỗ trợ IPv6 — lưu lượng đi ra qua NAT64/DNS64 của nhà cung cấp. Nếu git clone treo, kiểm tra cấu hình DNS64 theo tài liệu của họ.

Lưu ý

Kiểm tay PQC: `~/pqc/openssl/bin/openssl list -kem-algorithms | grep ML-KEM`. Chạy lại thấy “already installed ... Skipping” là *đúng thiết kế* (idempotency), muốn build lại thật thì `FORCE=1`.

7. build_liboqs.sh ref|opt — scripts/

Nhiệm vụ

Build liboqs thành **hai cây** với `OQS_DIST_BUILD=OFF` (“for use on a single machine”): cây ref (C thuần) và cây opt (bật tối ưu vi kiến trúc). **Đây là trái tim của WP3** — so hai cây trên cùng máy chính là đo “hiệu ứng tối ưu”.

Sinh ra gì & ở đâu

- `~/pqc/liboqs-ref/` hoặc `~/pqc/liboqs-neon/` (tùy tham số).
- Công cụ đo tốc độ: `~/pqc/src/liboqs/build-<variant>/tests/speed_kem` và `speed_sig`.
- `docs/liboqs.commit`.

Trên x86_64

```
bash scripts/build_liboqs.sh ref      # du de nhien thu chuoai liboqs
bash scripts/build_liboqs.sh opt      # TUY CHON: du lieu phu "hieu ung AVX2"
```

Trên x86, “opt” nghĩa là `-march=native` + tự dò bật **AVX2**/AES-NI. Chạy `speed_kem ML-KEM-768` đọc header:

- cây ref: CPU exts: SSE SSE2, AES: OpenSSL, `OQS_OPT_TARGET=generic`;
- cây opt: thêm `-march=native`, CPU exts: AVX2 ..., AES: NI, `OQS_OPT_TARGET=auto`.

► Số mẫu thật đã đo (LOQ, ML-KEM-768) — để biết “đúng” trông thế nào

Op	ref (µs)	opt (µs)	Speedup
keygen	30.985	10.705	×2.89
encaps	30.963	11.255	×2.75
decaps	36.250	13.718	×2.64

Gọi đúng tên: ×2.6–2.9 là “hiệu ứng tối ưu x86 trọn gói” (AVX2 đóng vai chính, kèm AES-NI), *không* viết “AVX2 nhanh ×2.9”.

Trên ARM Pi 4 (Mythic Beasts)

Bắt buộc CẢ HAI cây — đây mới là thí nghiệm NEON (RQ2):

```
bash scripts/build_liboqs.sh ref      # BAT BUOC
bash scripts/build_liboqs.sh opt      # BAT BUOC
```

Khác x86: ở đây “opt” nghĩa là bật **NEON** (`-DOQS_USE_ARM_NEON_INSTRUCTIONS=ON`) + `-mcpu=native`; cây ref nhận `NEON=OFF` tường minh (khối `uname -m` của script lần đầu thức dậy — trên x86 khối này im lặng). Đọc log `cmake` của cây ref phải thấy dòng `...NEON_INSTRUCTIONS=OFF` để tự xác nhận cây ref “sạch” NEON.

► Điểm phương pháp luận đặc biệt của Pi 4

Cortex-A72/BCM2711 **KHÔNG** có **ARM Crypto Extensions** (Broadcom không mua license) — nên AES/SHA2 phần cứng vắng mặt, các cờ đó tự-dò sẽ OFF ở cả hai cây. Hệ

quả tốt: cặp ref/opt càng “sạch”, chênh lệch đúng bằng NEON. (Trên Ampere/Graviton thì AES/SHA2 bất đều ở hai cây, delta vẫn chỉ là NEON.)

Lưu ý

- **PMU (đếm chu kỳ) trên Pi:** ARM không có rdtsc; muốn cycles chính xác cần nạp kernel module *trước* (clone mupq/pqax, vào enable_ccr, make install — cần sudo + linux headers) rồi mới build lại với USE_ARM_PMU=1 FORCE=1. Pi 4 là bare-metal nên làm được; nếu không, bỏ PMU, dùng wall-clock và ghi một dòng Limitations.
- **Throttling nhiệt:** Pi 4 giảm xung khi nóng (80–85°C). Giữa các batch đo phải để nguội; theo dõi bằng vcgencmd measure_temp (Pi OS) hoặc /sys/class/thermal.
- generic đổi nghĩa theo máy: x86 → -march=x86-64; ARM64 → -mcpu=cortex-a53. Nên “ref-x86” và “ref-ARM” là hai baseline khác nhau — cấm so chéo số thô.

8. fetch_pqclean.sh [clean|avx2|aarch64] — scripts/

Nhiệm vụ

Clone PQClean và build thư viện .a **tách riêng từng thuật toán**. Ba việc cho đồ án: (1) checkbox “reference implementations” của đề bài; (2) đo *code size từng thuật toán* cho WP5 — việc mà libcrypto.so/liboqs.so gộp không làm được; (3) trực đối chứng NEON thứ hai cho WP3.

Sinh ra gì & ở đâu

- ~/pqc/src/PQClean/crypto_kem|crypto_sign/<scheme>/<variant>/lib*.a (6 scheme: ml-kem-512/768/1024, ml-dsa-44/65/87).
- docs/pqclean.commit.

Trên x86_64

```
bash scripts/fetch_pqclean.sh clean      # ban portable C (moi may)
bash scripts/fetch_pqclean.sh avx2      # tuy chon: ban toi uu x86
```

Đo: size ~/pqc/src/PQClean/crypto_kem/ml-kem-768/{clean,avx2}/lib*.a. Bản avx2 thường *to hơn* clean — đó là “cái giá bộ nhớ của tốc độ”, một nhận định ăn điểm Analysis.

Trên ARM Pi 4 (Mythic Beasts)

```
bash scripts/fetch_pqclean.sh clean
bash scripts/fetch_pqclean.sh aarch64    # ban NEON: assembly viet tay
```

Cặp clean vs aarch64 trên cùng Pi là trực NEON độc lập với liboqs. Bản aarch64 của PQClean dùng **assembly NEON viết tay** (các file neon_*.o, __asm_NTT.o) — chi tiết đẹp để trích vào báo cáo.

Lưu ý

Script có **cổng chặn sai kiến trúc**: gõ aarch64 trên x86 (hoặc avx2 trên ARM) sẽ bị chặn ngay với một dòng lỗi rõ ràng — thấy lỗi đó là *đúng*, không phải hỏng. PQClean sắp được archive read-only (7/2026), kế nhiệm là PQ Code Package — ghi một câu vào phần Literature Review.

9. build_oqsprovider.sh [ref|opt] — scripts/

Nhiệm vụ

Build OQS Provider — kế nhiệm *chính thức* của fork “OQS-OpenSSL” mà đề bài nêu (fork đó đã ngừng phát triển). Provider này cắm thuật toán PQC vào OpenSSL cho nhánh thí nghiệm

TLS-qua-liboqs.

Sinh ra gì & ở đâu

~/pqc/oqs-provider/lib/oqsprovider.so và docs/oqsprovider.commit.

Trên cả hai máy

```
bash scripts/build_oqsprovider.sh opt # link với cây liboqs-neon (mac định)
bash scripts/build_oqsprovider.sh ref # hoặc link với cây liboqs-ref
```

Điều kiện trước (hai cổng gác): phải có OpenSSL và liboqs *đúng variant* bạn yêu cầu. Trên ARM, muốn provider-NEON thì phải build liboqs opt trước. Kiểm bằng:

```
openssl list -providers -provider-path ~/pqc/oqs-provider/lib -provider oqsprovider
```

phải thấy oqsprovider trong danh sách.

Lưu ý

Chỉ cần khi làm nhánh TLS bằng cây liboqs. WP2/WP4 với OpenSSL native **không** cần file này — chưa tới lúc đó thì chưa phải chạy.

10. cross_build_liboqs.sh — scripts/

Nhiệm vụ

Cross-compile liboqs cho aarch64 *ngay trên PC x86* — bằng chứng đáp ứng về “separate cross-compile scripts” của đề bài. Binary này **KHÔNG dùng để đo**.

Sinh ra gì & ở đâu

~/pqc/liboqs-aarch64-cross/ (chứa .so là mã ARM generic).

Trên x86_64

```
sudo apt install gcc-aarch64-linux-gnu g++-aarch64-linux-gnu # một lần
bash scripts/cross_build_liboqs.sh
```

Kiểm: dòng cuối file .../liboqs.so phải in “ARM aarch64”. Đây là phép “thợ may đo ni cho người vắng mặt”: compiler x86 tạo ra mã ARM nhưng không chạy thử tại chỗ được.

Trên ARM Pi 4

Không dùng — đã đứng trên máy ARM thật thì build native (mục 7) vừa đúng vừa cho số đo thật.

Lưu ý

Vai trò chỉ là *Build & Reproducibility*. Tuyệt đối không lấy binary cross này ra đo rồi đưa số vào báo cáo.

11. aarch64-toolchain.cmake — cmake/

Nhiệm vụ

Định nghĩa “toolchain” để CMake biết cross-compile x86→aarch64: đặt CMAKE_SYSTEM_NAME, SYSTEM_PROCESSOR aarch64, trở CMAKE_C_COMPILER tới aarch64-linux-gnu-gcc, và bộ bốn FIND_ROOT_PATH_MODE*. Đồng mẫu với file toolchain_rasppi.cmake mà liboqs ship.

Sinh ra gì & ở đâu

Không sinh ra gì. Là file cấu hình.

Trên cả hai máy

Không bao giờ chạy trực tiếp — nó được `cross_build_liboqs.sh` nạp qua `-DCMAKE_TOOLCHAIN_FILE`. Chỉ có ý nghĩa trên x86 (đầu vào cho `cross`). Kiểm cú pháp: `cmake -P cmake/aarch64-toolchain.cmake` chạy im lặng là OK.

12. `docker_buildx.sh` — `scripts/`

Nhiệm vụ

Đóng hộp môi trường cho **cả hai kiến trúc cùng lúc**, đứng trên máy x86: `buildx` chạy cùng một Dockerfile hai lần — lần amd64 build thường, lần arm64 build trong **QEMU** (phần mềm giả lập CPU ARM). Bằng chứng về “`buildx multiarch`” của đề bài.

Sinh ra gì & ở đâu

Image `nt219-pqc:multi` (hai-trong-một) trong Docker.

Trên x86_64

```
docker run --privileged --rm tonistiigi/binfmt --install all # cai QEMU, mot lan
bash scripts/docker_buildx.sh
```

Trên ARM Pi 4

Không cần — image bản arm64 đã được sinh sẵn từ x86 qua `buildx`; nếu cần môi trường đóng hộp trên Pi thì chỉ việc `docker run` image đó.

Lưu ý

Cấm đo trong QEMU. Số đo bên trong container arm64 chạy trên PC là tốc độ của *trình giả lập*, không phải của Cortex-A72 — vô nghĩa khoa học. Caveat kỹ thuật: cờ `--load` có thể từ chối image đa-platform (“does not support manifest lists”); khi đó bật `containerd image store` hoặc build tách từng `--platform`.

13. `Dockerfile.x86_64` — `docker/`

Nhiệm vụ

Mô tả môi trường đóng hộp — yêu cầu *nguyên văn* của đề bài (“Provide Dockerfile for `x86_64`”). Điểm hay: thay vì chép lại lệnh build, nó **tái dùng chính các script của repo** (`build_openssl.sh`, `make`, `verify_env.sh`) — một nguồn sự thật; cổng `verify_env` ở cuối làm image *tự fail* nếu môi trường hỏng.

Sinh ra gì & ở đâu

Image `nt219-pqc` trong Docker.

Trên x86_64

```
docker build -f docker/Dockerfile.x86_64 -t nt219-pqc . # chạy tu GOC repo
docker run -it --rm nt219-pqc
```

Trong container, `openssl version` phải ra 3.6.2.

Trên ARM Pi 4

Bản ARM của image đến từ `docker_buildx.sh` (qua QEMU), không build riêng trên Pi.

Lưu ý

Phải chạy `docker build` từ **gốc repo** (để context . nhìn thấy `scripts/` và `src/` mà COPY). Pin phiên bản qua biến + bỏ test suite trong CI giống quy ước của `oqs-demos`.

14. verify_env.sh — scripts/

Nhiệm vụ

Hai vai: (1) *cổng chống crypto-failure* — chặn lại nếu OpenSSL đang dùng thiếu ML-KEM/ML-DSA; (2) *biên bản môi trường* — ghi gcc, cờ build, CPU governor. Trả lời trực tiếp câu “Document compiler versions, flags, and CPU governor settings” của đề bài.

Sinh ra gì & ở đâu

docs/env_report_<arch>.txt (tên file tự gán kiến trúc).

Trên x86_64

bash scripts/verify_env.sh → docs/env_report_x86_64.txt. Thấy PASS. Report written to ... là đạt.

Trên ARM Pi 4 (Mythic Beasts)

Cùng lệnh → docs/env_report_aarch64.txt. **Phải chạy trên cả hai máy** để nộp kèm hai biên bản. Governor trên Pi mặc định ondemand (dao động xung) — trước khi đo nên đặt performance:

```
sudo cpupower frequency-set -g performance
# nếu thiếu cpupower:
echo performance | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
```

Lưu ý

Cần OpenSSL đã build trước (thiếu thì chính nó là cái fail đầu tiên, kèm chỉ dẫn). Chạy lại sau mỗi lần đổi môi trường để biên bản tự cập nhật.

15. Hồ sơ tái lập (thư mục docs/)

Sáu mảnh bằng chứng tự sinh khi chạy, nộp kèm báo cáo: bốn file *.commit (commit chính xác của OpenSSL/liboqs/PQClean/oqs-provider) cộng hai env_report_<arch>.txt (compiler, cờ, governor của từng máy). Không phải gõ tay — chúng xuất hiện như sản phẩm phụ của quá trình build, đúng tinh thần “ai cũng dựng lại được y hệt”.

16. Quy tắc bốn con số khi so sánh

Sau hai máy, riêng liboqs ML-KEM-768 bạn có bốn số. Dùng chúng cho đúng:

Phép so	Hợp lệ?	Trả lời câu gì
ref-x86 ↔ opt-x86 (cùng máy)	Có	hiệu ứng AVX2/tối ưu x86 (đã đo ×2.6–2.9)
ref-ARM ↔ opt-ARM (cùng máy)	Có	hiệu ứng NEON — chính là RQ2
opt-x86 ↔ opt-ARM (số thô)	Không	đổi 2 biến cùng lúc (CPU + loại tối ưu)
(opt/ref) _{x86} ↔ (opt/ref) _{ARM}	Có	“tối ưu đáng giá ×A trên server, ×B trên Pi”

Cách “bắc cầu” giữa hai máy là so *tỉ lệ* (đại lượng không thứ nguyên, đã tự chuẩn hóa theo máy), không bao giờ so số thô. Cùng logic áp cho bench_ev (RQ1): PQC-vs-RSA so trong từng máy, chéo máy chỉ so *tỉ số* PQC/RSA.

17. Phụ lục — Trình tự chạy đầy đủ, hai máy

Trên PC x86_64

```

cd <repo> && chmod +x scripts/*.sh
bash scripts/install_prereqs.sh          # 1) toolchain + cong cu do (sudo)
bash scripts/build_openssl.sh            # 2) OpenSSL 3.6.2 PQC native
source scripts/setenv.sh                 # 3) bat moi truong (MOI terminal)
bash scripts/verify_env.sh               # 4) cong gac + bien ban
make                                     # 5) build bench_evp (WP2)
bash scripts/build_liboqs.sh ref          # 6) liboqs ref (nghiem thu)
bash scripts/fetch_pqclean.sh clean      # 7) PQClean (WP5 code-size)
# Tuy chon: build_liboqs.sh opt | fetch_pqclean.sh avx2
# Bang chung $7.3 (chi x86): cross_build_liboqs.sh | docker build ...

```

Trên ARM — Raspberry Pi 4 (Mythic Beasts)

```

git clone <repo> && cd <repo> && chmod +x scripts/*.sh
bash scripts/install_prereqs.sh
SKIP_TESTS=1 bash scripts/build_openssl.sh # Pi build cham; full test qua dem 1 lan
source scripts/setenv.sh
bash scripts/verify_env.sh                 # -> docs/env_report_aarch64.txt
sudo cpupower frequency-set -g performance # co dinh xung truoc khi do
make
bash scripts/build_liboqs.sh ref           # BAT BUOC ca hai cay = thi nghiem NEON
bash scripts/build_liboqs.sh opt
bash scripts/fetch_pqclean.sh clean
bash scripts/fetch_pqclean.sh aarch64
# Luu y Pi 4: nhiet do (de nguoi giua cac batch), PMU can module pqax,
# mang IPv6-only (clone qua NAT64), PUSH du lieu truoc khi tra may.

```

*Mọi binary trong ~/pqc build trên máy nào chỉ dùng trên máy đó.
Số đo là tài sản theo máy — so trực tiếp trong cùng máy, bắc cầu giữa máy bằng tỉ lệ.*